گزارش پروژه شناسایی حروف با استفاده از شبکه عصبی پرسپترون

۱ .هدف پروژه

هدف این پروژه پیاده‌سازی شبکه عصبی پرسپترون برای تشخیص حروف انگلیسی است. در این پروژه سه روش مختلف پیاده‌سازی و بررسی شده‌اند:

1. تشخیص دودویی حرف A از سایر حروف با استفاده از یک شبکه پرسپترون تک خروجی.
2. تشخیص هفت حرفA ، B، C، D، E، J و K با استفاده از یک شبکه پرسپترون چند خروجی.
3. تشخیص همان هفت حرف با استفاده از هفت شبکه پرسپترون مستقل.

در نهایت، عملکرد این سه روش با یکدیگر مقایسه شده و شبکه‌ها روی تعدادی تصویر که در مرحله آموزش استفاده نشده‌اند، آزمایش شده‌اند.

۲ .ساختار شبکه و نحوه نمایش ورودی‌ها و خروجی‌ها

۲-۱ .نمایش ورودی‌ها

برای ورود تصاویر به شبکه عصبی، هر تصویر به یک ماتریس ۹ در ۷ تبدیل شده است. بنابراین هر تصویر شامل ۶۳ پیکسل است.

برای تبدیل تصویر به ورودی قابل استفاده در شبکه، هر پیکسل به یکی از مقادیر زیر نگاشت می‌شود:

- پیکسل با مقدار # به عدد ۱ تبدیل می‌شود.
- پیکسل با مقدار . به عدد ۱- تبدیل می‌شود.
- سایر کاراکترها به عدد صفر تبدیل می‌شوند.

در نتیجه، هر تصویر به یک بردار شامل ۶۳ مقدار تبدیل می‌شود که به‌عنوان ورودی شبکه عصبی استفاده می‌شود.

این شیوه نمایش باعث می‌شود اطلاعات هر تصویر به شکل عددی در اختیار شبکه قرار گیرد و شبکه بتواند با استفاده از وزن‌های خود الگوهای مربوط به هر حرف را یاد بگیرد.

۲-۲ .شبکه پرسپترون تک‌خروجی (مرحله اول)

در مرحله اول، هدف تشخیص حرف A از سایر حروف است.

ساختار شبکه شامل موارد زیر است:

- ۶۳ ورودی، متناظر با ۶۳ پیکسل تصویر
- یک نورون خروجی
- یک بایاس
- ۶۳ وزن مربوط به ورودی‌ها

خروجی شبکه با استفاده از مجموع وزنی ورودی‌ها و بایاس محاسبه می‌شود:

$$y_{in} = \sum_{i=0}^{62} x_i w_i + b$$

تابع فعال‌سازی شبکه با آستانه ۰/۲ تعریف شده است:

- اگر خروجی بزرگ‌تر از ۰/۲ باشد، نتیجه برابر با ۱ است.
- اگر خروجی کوچک‌تر از ۰/۲- باشد، نتیجه برابر با ۱- است.
- در غیر این صورت، خروجی صفر خواهد بود.

در این پروژه، خروجی ۱ به معنی تشخیص حرف A و خروجی ۱- به معنی تشخیص سایر حروف است. خروجی صفر نیز به‌عنوان وضعیت نامطمئن در نظر گرفته می‌شود.

۲-۳ .شبکه پرسپترون چندخروجی (مرحله دوم)

در مرحله دوم، هدف تشخیص هفت حرف A ، B، C، D، E، J و K است.

ساختار این شبکه شامل:

- ۶۳ ورودی
- ۷ نورون خروجی
- ۷ مجموعه وزن، هرکدام شامل ۶۳ وزن
- ۷ مقدار بایاس

هر نورون خروجی مسئول تشخیص یک حرف است.

برای نمایش خروجی مورد انتظار، از مقادیر دوقطبی ۱ و ۱- استفاده شده است. برای مثال، اگر ورودی مربوط به حرف Aباشد، بردار هدف به شکل زیر خواهد بود:

[1, -1, -1, -1, -1, -1, -1]

برای حرف B ، بردار هدف به شکل زیر است:

[-1, 1, -1, -1, -1, -1, -1]

این روند برای سایر حروف نیز به همین شکل انجام می‌شود. در هر بردار هدف، فقط موقعیت مربوط به حرف موردنظر مقدار ۱ دارد و سایر موقعیت‌ها برابر با ۱- هستند.

در زمان پیش‌بینی، خروجی هر هفت نورون محاسبه می‌شود. اگر دقیقاً یک خروجی برابر ۱ و شش خروجی دیگر برابر ۱- باشند، حرف متناظر تشخیص داده می‌شود. در غیر این صورت، نتیجه به‌عنوان نامشخص در نظر گرفته می‌شود.

۲-۴ .هفت پرسپترون مستقل (مرحله سوم)

در مرحله سوم، به‌جای استفاده از یک شبکه با هفت خروجی، از هفت شبکه پرسپترون مستقل استفاده شده است.

هر شبکه مسئول تشخیص یک حرف است. برای مثال:

- شبکه اول: تشخیص A از سایر حروف
- شبکه دوم: تشخیص B از سایر حروف
- شبکه سوم: تشخیص C از سایر حروف
- شبکه چهارم: تشخیص D از سایر حروف
- شبکه پنجم: تشخیص E از سایر حروف
- شبکه ششم: تشخیص J از سایر حروف
- شبکه هفتم: تشخیص K از سایر حروف

در هر شبکه، نمونه‌های مربوط به حرف موردنظر دارای هدف ۱ و نمونه‌های مربوط به سایر حروف دارای هدف ۱- هستند.

در زمان آزمایش، تصویر ورودی به هر هفت شبکه داده می‌شود. اگر فقط یکی از شبکه‌ها خروجی ۱ و شش شبکه دیگر خروجی ۱- داشته باشند، حرف مربوطه تشخیص داده می‌شود. در غیر این صورت، نتیجه نامشخص خواهد بود.

۳ .پیاده‌سازی و قانون یادگیری

۳-۱ .پیاده‌سازی شبکه

پیاده‌سازی پروژه با زبان Python انجام شده و بخش‌های مختلف برنامه در فایل‌های جداگانه قرار گرفته‌اند.

فایل perceptron.py شامل کلاس پرسپترون پایه است. این کلاس وظیفه نگهداری وزن‌ها، بایاس، محاسبه ورودی خالص نورون، محاسبه خروجی و به‌روزرسانی وزن‌ها را بر عهده دارد.

فایل MultiOutputPerceptron.py برای پیاده‌سازی شبکه دارای هفت خروجی استفاده شده است. در این کلاس، برای هر نورون خروجی یک مجموعه وزن و یک بایاس مستقل در نظر گرفته شده است.

فایل IndependentPerceptrons.py شامل کلاس هفت پرسپترون مستقل است. این کلاس با استفاده از کلاس پرسپترون پایه، هفت شبکه جداگانه ایجاد و آموزش می‌دهد.

همچنین فایل file_handling.py برای خواندن فایل‌های ورودی، استخراج الگوهای آموزشی و تبدیل کاراکترهای تصویر به مقادیر عددی استفاده شده است.

۳-۲ .قانون یادگیری پرسپترون

در این پروژه از قانون یادگیری پرسپترون استفاده شده است.

در ابتدای آموزش، تمام وزن‌ها برابر صفر در نظر گرفته شده‌اند. نرخ یادگیری نیز برابر با ۱ و آستانه تابع فعال‌سازی برابر با ۰/۲ است.

در هر مرحله، ابتدا خروجی شبکه برای نمونه آموزشی محاسبه می‌شود. سپس خروجی محاسبه‌شده با مقدار هدف مقایسه می‌شود.

اگر خروجی شبکه با مقدار هدف متفاوت باشد، وزن‌ها و بایاس به‌روزرسانی می‌شوند.

قانون به‌روزرسانی وزن‌ها به شکل زیر است:

$$w_i(new)=w_i(old)+ alpha x_i t$$

قانون به‌روزرسانی بایاس نیز به شکل زیر است:

$$b(new)=b(old)+ alpha t$$

در این روابط:

- w_i وزن ورودی شماره i
- x_i مقدار ورودی شماره i
- t مقدار هدف
- alpha نرخ یادگیری

در این پروژه مقدار نرخ یادگیری برابر با ۱ است.

این فرایند برای تمام نمونه‌های آموزشی تکرار می‌شود. یک دور کامل پردازش تمام نمونه‌های آموزشی، یک Epoch نامیده می‌شود.

آموزش تا زمانی ادامه پیدا می‌کند که در یک دور کامل، دیگر تغییری در وزن‌ها ایجاد نشود یا تعداد Epoch ها به حداکثر مقدار تعیین‌شده، یعنی ۱۰۰۰، برسد.

در شبکه چندخروجی، همین قانون یادگیری برای هر هفت نورون خروجی به‌صورت مستقل اعمال می‌شود.

۴ .نتایج آموزش شبکه

پس از اجرای برنامه، وزن‌های نهایی شبکه‌ها محاسبه و نمایش داده شدند.

در خروجی اجرای برنامه، برای شبکه پرسپترون چندخروجی پیام زیر مشاهده شد:

Training converged after 6 epochs.

این نتیجه نشان می‌دهد که شبکه چندخروجی پس از ۶ دور آموزش به وضعیتی رسیده است که در آن، در یک دور کامل دیگر نیازی به به‌روزرسانی وزن‌ها نبوده است.

وزن‌های نهایی هر هفت نورون خروجی نمایش داده شدند. همچنین وزن‌های هفت پرسپترون مستقل نیز پس از آموزش استخراج شدند.

وزن‌های مربوط به حروف A و B در شبکه چندخروجی و شبکه‌های مستقل یکسان هستند. وزن‌های مربوط به سایر حروف نیز در خروجی دو روش متناظر یکسان مشاهده شدند.

این موضوع نشان می‌دهد که در اجرای انجام‌شده، هر دو روش برای هر حرف به وزن‌های یکسانی رسیده‌اند.

۵ .نتایج آموزش شبکه‌ها

پس از اجرای برنامه، شبکه‌های عصبی آموزش داده شدند و وزن‌های نهایی آن‌ها محاسبه شد.

در مرحله دوم، شبکه چندخروجی پس از ۶ دوره آموزش به همگرایی رسید. در این مرحله، هر خروجی مسئول تشخیص یکی از حروف A ، B، C، D، E، J و K بود.

در مرحله سوم نیز برای هر حرف یک پرسپترون مستقل در نظر گرفته شد. این پرسپترون‌ها با استفاده از داده‌های آموزشی مربوط به همان حرف آموزش داده شدند.

۶ .بررسی نتایج روی داده‌های تست

برای بررسی عملکرد شبکه‌ها، از یک فایل تست شامل ۲۱ تصویر استفاده شد. تصاویر به ترتیب شامل حروف A ، B، C، D، E، J و K بودند که این ترتیب سه بار تکرار شده بود.

هر تصویر به یک بردار ورودی ۶۳ عضوی تبدیل شد و سپس به شبکه‌های عصبی داده شد.

نتایج حاصل از اجرای سه مرحله در جدول زیر آمده است:

شماره تصویر	حرف واقعی	مرحله اول	مرحله دوم	مرحله سوم
۱	A	A	A	A
۲	B	Not A	B	B
۳	C	Not A	نامشخص	نامشخص
۴	D	Not A	D	D
۵	E	Not A	E	E
۶	J	Not A	J	J
۷	K	Not A	K	K
۸	A	A	نامشخص	نامشخص

۹	B	Not A	B	B
۱۰	C	Not A	C	C
۱۱	D	Not A	D	D
۱۲	E	Not A	E	E
۱۳	J	Not A	نامشخص	نامشخص
۱۴	K	Not A	K	K
۱۵	A	A	A	A
۱۶	B	Not A	نامشخص	نامشخص
۱۷	C	Not A	C	C
۱۸	D	Not A	نامشخص	نامشخص
۱۹	E	Not A	نامشخص	نامشخص
۲۰	J	Not A	نامشخص	نامشخص
۲۱	K	Not A	K	K

در مرحله دوم و سوم، اگر شبکه نتواند دقیقاً یک خروجی مثبت و شش خروجی منفی تولید کند، نتیجه به‌صورت «نامشخص» در نظر گرفته می‌شود.

۷ .محاسبه دقت تشخیص

با توجه به اینکه حروف واقعی تصاویر تست مشخص هستند، می‌توان تعداد تشخیص‌های صحیح و اشتباه هر مرحله را محاسبه کرد.

مرحله اول

هدف این مرحله تشخیص حرف A از سایر حروف بود.

از مجموع ۲۱ تصویر، سه تصویر مربوط به حرف A و ۱۸ تصویر مربوط به سایر حروف بودند.

- تشخیص صحیح A: سه مورد
- تشخیص صحیح سایر حروف (Not A): هجده مورد
- تشخیص اشتباه: صفر مورد

بنابراین، دقت تشخیص در مرحله اول برابر است با:

Accuracy = 21 / 21 × 100 = 100%

مرحله دوم

در این مرحله شبکه باید یکی از هفت حرف را تشخیص می‌داد.

از مجموع ۲۱ تصویر، شبکه توانست ۱۴ تصویر را به‌درستی تشخیص دهد و در ۷ تصویر نتیجه نامشخص تولید کرد.

حرف	تعداد تصاویر	تشخیص صحیح	تشخیص اشتباه یا نامشخص
A	۳	۲	۱
B	۳	۲	۱
C	۳	۲	۱
D	۳	۲	۱
E	۳	۲	۱
J	۳	۱	۲
K	۳	۳	۰
مجموع	**۲۱**	**۱۴**	**۷**

بنابراین، دقت کلی شبکه در مرحله دوم برابر است با:

Accuracy = 14 / 21 × 100 ≈ 66.67%

بیشترین عملکرد مربوط به حرف K بود که هر سه نمونه آن به‌درستی تشخیص داده شدند. در مقابل، حرف J با یک تشخیص صحیح از سه نمونه، کمترین دقت را داشت.

مرحله سوم

در مرحله سوم نیز از هفت پرسپترون مستقل برای تشخیص حروف استفاده شد.

نتایج این مرحله دقیقاً مشابه مرحله دوم بود؛ یعنی از ۲۱ تصویر، ۱۴ تصویر به‌درستی تشخیص داده شدند و ۷ تصویر با خروجی نامشخص مواجه شدند.

بنابراین، دقت کلی مرحله سوم نیز برابر با **66.67 درصد** است.

۸ .بررسی خطاهای تشخیص

در مراحل دوم و سوم، هفت تصویر به‌درستی شناسایی نشدند. در تمام این موارد، خروجی شبکه به‌صورت نامشخص بود؛ یعنی شبکه نتوانست یک حرف مشخص را با شرایط تعیین‌شده تشخیص دهد.

این اتفاق می‌تواند به دلیل شباهت ساختاری حروف، نحوه توزیع وزن‌ها یا محدودیت توانایی پرسپترون در تفکیک الگوهای ورودی رخ دهد.

همچنین، چون ورودی شبکه به یک ماتریس ۹×۷ تبدیل شده و سپس به یک بردار ۶۳ عضوی تبدیل می‌شود، تغییر در اندازه، ضخامت، موقعیت یا شکل ظاهری حروف می‌تواند روی نتیجه تشخیص تأثیر بگذارد.

۹ .مقایسه مراحل دوم و سوم

در مرحله دوم، از یک شبکه عصبی با هفت خروجی استفاده شد. هر خروجی متناظر با یکی از حروف موردنظر بود و در زمان آموزش، خروجی صحیح مقدار مثبت و سایر خروجی‌ها مقدار منفی دریافت می‌کردند.

در مرحله سوم، به‌جای یک شبکه با هفت خروجی، هفت پرسپترون مستقل ایجاد شد. هر پرسپترون وظیفه داشت یک حرف را از سایر حروف تشخیص دهد.

نتایج تست نشان داد که خروجی دو روش برای تمام ۲۱ تصویر یکسان بوده است.

میزان تطابق دو روش: 21 / 21 × 100 = 100%

همچنین، دقت تشخیص هر دو روش برابر با 66.67 درصد به دست آمد.

از نظر پیاده‌سازی، روش دوم از یک کلاس شبکه چندخروجی استفاده می‌کند، درحالی‌که در روش سوم هفت نمونه مستقل از کلاس پرسپترون ساخته می‌شود. با وجود تفاوت در ساختار پیاده‌سازی، نتایج حاصل از دو روش در این آزمایش یکسان بودند.

۱۰ .بررسی عملکرد روی تصاویر جدید

برای ارزیابی عملکرد شبکه‌ها، ۲۱ تصویر در فایل تست قرار داده شد که شامل سه نمونه از هر یک از حروف A ، B، C، D، E، J و K بود.

نتایج نشان داد که شبکه تک‌خروجی در تشخیص A از سایر حروف عملکرد بسیار خوبی داشت و تمام تصاویر را به‌درستی طبقه‌بندی کرد.

در مقابل، شبکه چندخروجی و شبکه‌های مستقل در تشخیص هفت حرف عملکرد ضعیف‌تری داشتند و در برخی نمونه‌ها نتوانستند حرف موردنظر را تشخیص دهند.

این نتایج نشان می‌دهد که تشخیص یک حرف از سایر حروف، در این مجموعه داده، ساده‌تر از تفکیک هفت حرف از یکدیگر بوده است.

۱۱ .نتیجه‌گیری

در این پروژه، سه روش مختلف مبتنی بر پرسپترون برای تشخیص حروف الفبا پیاده‌سازی و بررسی شدند.

در مرحله اول، شبکه توانست حرف A را با دقت ۱۰۰ درصد از سایر حروف تشخیص دهد.

در مرحله دوم، از یک شبکه عصبی با هفت خروجی استفاده شد که دقت آن در تشخیص حروف برابر با 66.67 درصد بود.

در مرحله سوم، هفت پرسپترون مستقل پیاده‌سازی شدند که عملکرد آن‌ها دقیقاً مشابه شبکه چندخروجی بود و دقت 66.67 درصد به دست آمد.

در مجموع، نتایج نشان داد که هر دو روش چندخروجی و پرسپترون‌های مستقل در این آزمایش عملکرد یکسانی داشتند. همچنین، عملکرد بهتر مرحله اول نشان داد که تشخیص دودویی یک حرف از سایر حروف نسبت به تشخیص هم‌زمان چند حرف، مسئله ساده‌تری برای شبکه مورد استفاده بوده است.

خروجی کامل برنامه:

this is the weights after teaching the neural network:

-2(x0) + -2(x1) + -2(x2) + 0(x3) + -2(x4) + -2(x5) + 0(x6) + 2(x7) + -4(x8) + 2(x9) + 4(x10) + 0(x11) + 0(x12) + 0(x13) + 0(x14) + -2(x15) + 2(x16) + 2(x17) + 2(x18) + 2(x19) + -2(x20) + 0(x21) + -2(x22) + 2(x23) + 2(x24) + 4(x25) + 2(x26) + 0(x27) + 0(x28) + -2(x29) + 0(x30) + 0(x31) + 2(x32) + 0(x33) + 2(x34) + 0(x35) + 0(x36) + 4(x37) + 2(x38) + 4(x39) + 4(x40) + 0(x41) + 0(x42) + 0(x43) + 2(x44) + 2(x45) + 0(x46) + 4(x47) + 0(x48) + 2(x49) + -2(x50) + 2(x51) + 2(x52) + 2(x53) + 0(x54) + 0(x55) + 0(x56) + 0(x57) + -2(x58) + -2(x59) + 0(x60) + 0(x61) + 2(x62) + -2

Training converged after 6 epochs.

Weights after training Multi-Output Perceptron:

Output neuron for A:

-2(x0) + -2(x1) + -2(x2) + 0(x3) + -2(x4) + -2(x5) + 0(x6) + 2(x7) + -4(x8) + 2(x9) + 4(x10) + 0(x11) + 0(x12) + 0(x13) + 0(x14) + -2(x15) + 2(x16) + 2(x17) + 2(x18) + 2(x19) + -2(x20) + 0(x21) + -2(x22) + 2(x23) + 2(x24) + 4(x25) + 2(x26) + 0(x27) + 0(x28) + -2(x29) + 0(x30) + 0(x31) + 2(x32) + 0(x33) + 2(x34) + 0(x35) + 0(x36) + 4(x37) + 2(x38) + 4(x39) + 4(x40) + 0(x41) + 0(x42) + 0(x43) + 2(x44) + 2(x45) + 0(x46) + 4(x47) + 0(x48) + 2(x49) + -2(x50) + 2(x51) + 2(x52) + 2(x53) + 0(x54) + 0(x55) + 0(x56) + 0(x57) + -2(x58) + -2(x59) + 0(x60) + 0(x61) + 2(x62) + -2

Output neuron for B:

2(x0) + 2(x1) + -2(x2) + -2(x3) + 0(x4) + 4(x5) + -2(x6) + 2(x7) + 0(x8) + 2(x9) + 0(x10) + 2(x11) + -2(x12) + 4(x13) + 0(x14) + 2(x15) + 2(x16) + 0(x17) + 2(x18) + 2(x19) + 4(x20) + 0(x21) + 2(x22) + 2(x23) + 2(x24) + 2(x25) + 4(x26) + 2(x27) + 0(x28) + 2(x29) + 2(x30) + 4(x31) + 4(x32) + 6(x33) + 0(x34) + 0(x35) + 0(x36) + 0(x37) + -2(x38) + 0(x39) + 0(x40) + 4(x41) + 0(x42) + 0(x43) + 2(x44) + 2(x45) + 2(x46) + 0(x47) + 4(x48) + 2(x49) + -2(x50) + 2(x51) + 2(x52) + 2(x53) + -4(x54) + 4(x55) + 0(x56) + 0(x57) + -2(x58) + 0(x59) + -2(x60) + 2(x61) + -2(x62) + -2

Output neuron for C:

-3(x0) + -1(x1) + 1(x2) + -1(x3) + 1(x4) + -1(x5) + 3(x6) + -1(x7) + 5(x8) + 3(x9) + 1(x10) + 1(x11) + 1(x12) + 3(x13) + 3(x14) + 1(x15) + 3(x16) + -1(x17) + 3(x18) + 1(x19) + 1(x20) + 3(x21) + 1(x22) + -1(x23) + 1(x24) + 1(x25) + 1(x26) + 1(x27) + 3(x28) + -3(x29) + -3(x30) + -1(x31) + -1(x32) + -1(x33) + 3(x34) + 3(x35) + -1(x36) + -1(x37) + -1(x38) + 1(x39) + -1(x40) + 1(x41) + 3(x42) + -3(x43) + 3(x44) + 1(x45) + 3(x46) + -1(x47) + 1(x48) + -1(x49) + 1(x50) + 3(x51) + 3(x52) + 1(x53) + -1(x54) + 3(x55) + -5(x56) + -3(x57) + -1(x58) + 1(x59) + -1(x60) + -1(x61) + -1(x62) + -3

Output neuron for D:

1(x0) + 1(x1) + -1(x2) + -1(x3) + 1(x4) + -5(x5) + -1(x6) + 1(x7) + 3(x8) + 3(x9) + 1(x10) + 3(x11) + 3(x12) + -1(x13) + 1(x14) + 3(x15) + 3(x16) + 1(x17) + 1(x18) + 1(x19) + 3(x20) + 1(x21) + 3(x22) + 1(x23) + 1(x24) + 1(x25) + 1(x26) + 3(x27) + 1(x28) + -1(x29) + -5(x30) + -1(x31) + -3(x32) + -3(x33) + 7(x34) + 1(x35) + 1(x36) + 1(x37) + -1(x38) + 1(x39) + -1(x40) + 3(x41) + 1(x42) + -1(x43) + 3(x44) + 3(x45) + 1(x46) + -1(x47) + 3(x48) + 1(x49) + -1(x50) + 3(x51) + 3(x52) + 3(x53) + 1(x54) + -1(x55) + -1(x56) + -1(x57) + -3(x58) + 1(x59) + -1(x60) + -5(x61) + -1(x62) + -3

Output neuron for E:

3(x0) + 3(x1) + -1(x2) + 3(x3) + 5(x4) + -1(x5) + 7(x6) + 3(x7) + 1(x8) + 3(x9) + 1(x10) + -1(x11) + -1(x12) + -1(x13) + 1(x14) + 3(x15) + 3(x16) + -3(x17) + 1(x18) + 1(x19) + -5(x20) + 1(x21) + 3(x22) + -3(x23) + 7(x24) + 1(x25) + 1(x26) + -5(x27) + 1(x28) + 3(x29) + 1(x30) + 9(x31) + 1(x32) + -7(x33) + 3(x34) + 1(x35) + 1(x36) + 1(x37) + 1(x38) + 1(x39) + -1(x40) + -5(x41) + 1(x42) + -1(x43) + 3(x44) + 3(x45) + -3(x46) + -1(x47) + -5(x48) + 3(x49) + -3(x50) + 3(x51) + 3(x52) + 3(x53) + -7(x54) + -1(x55) + 1(x56) + 1(x57) + -3(x58) + 5(x59) + 3(x60) + -1(x61) + 9(x62) + -3

Output neuron for J:

0(x0) + 0(x1) + -4(x2) + -2(x3) + 0(x4) + 2(x5) + 4(x6) + 0(x7) + 0(x8) + 2(x9) + 0(x10) + 2(x11) + 2(x12) + 0(x13) + -2(x14) + 2(x15) + 2(x16) + 0(x17) + 2(x18) + 4(x19) + -2(x20) + -2(x21) + 2(x22) + 0(x23) + 2(x24) + 0(x25) + 4(x26) + 0(x27) + -2(x28) + 0(x29) + -2(x30) + 0(x31) + -2(x32) + 2(x33) + 2(x34) + -2(x35) + 0(x36) + 0(x37) + 0(x38) + 0(x39) + 2(x40) + 0(x41) + -2(x42) + 2(x43) + 2(x44) + 2(x45) + 2(x46) + 2(x47) + 0(x48) + 0(x49) + 0(x50) + 2(x51) + 2(x52) + 2(x53) + 0(x54) + 0(x55) + -2(x56) + -2(x57) + -2(x58) + 0(x59) + -2(x60) + -2(x61) + 0(x62) + -2

Output neuron for K:

2(x0) + 0(x1) + -2(x2) + -4(x3) + -2(x4) + 2(x5) + 2(x6) + 0(x7) + 2(x8) + 0(x9) + -2(x10) + 4(x11) + 0(x12) + -2(x13) + 0(x14) + 2(x15) + 0(x16) + 2(x17) + 0(x18) + 0(x19) + -2(x20) + 0(x21) + 2(x22) + 2(x23) + 0(x24) + -2(x25) + 0(x26) + -2(x27) + 0(x28) + 2(x29) + -2(x30) + -2(x31) + -4(x32) + -2(x33) + 0(x34) + 0(x35) + 0(x36) + 0(x37) + 0(x38) + -2(x39) + -2(x40) + -2(x41) + 0(x42) + 0(x43) + 0(x44) + 2(x45) + 2(x46) + -2(x47) + -2(x48) + 0(x49) + 0(x50) + 0(x51) + 0(x52) + 2(x53) + 0(x54) + -2(x55) + 0(x56) + -2(x57) + -2(x58) + -2(x59) + -4(x60) + 0(x61) + 0(x62) + 0

Weights after training seven independent perceptrons:

Network for letter A:

-2(x0) + -2(x1) + -2(x2) + 0(x3) + -2(x4) + -2(x5) + 0(x6) + 2(x7) + -4(x8) + 2(x9) + 4(x10) + 0(x11) + 0(x12) + 0(x13) + 0(x14) + -2(x15) + 2(x16) + 2(x17) + 2(x18) + 2(x19) + -2(x20) + 0(x21) + -2(x22) + 2(x23) + 2(x24) + 4(x25) + 2(x26) + 0(x27) + 0(x28) + -2(x29) + 0(x30) + 0(x31) + 2(x32) + 0(x33) + 2(x34) + 0(x35) + 0(x36) + 4(x37) + 2(x38) + 4(x39) + 4(x40) + 0(x41) + 0(x42) + 0(x43) + 2(x44) + 2(x45) + 0(x46) + 4(x47) + 0(x48) + 2(x49) + -2(x50) + 2(x51) + 2(x52) + 2(x53) + 0(x54) + 0(x55) + 0(x56) + 0(x57) + -2(x58) + -2(x59) + 0(x60) + 0(x61) + 2(x62) + -2

Network for letter B:

2(x0) + 2(x1) + -2(x2) + -2(x3) + 0(x4) + 4(x5) + -2(x6) + 2(x7) + 0(x8) + 2(x9) + 0(x10) + 2(x11) + -2(x12) + 4(x13) + 0(x14) + 2(x15) + 2(x16) + 0(x17) + 2(x18) + 2(x19) + 4(x20) + 0(x21) + 2(x22) + 2(x23) + 2(x24) + 2(x25) + 4(x26) + 2(x27) + 0(x28) + 2(x29) + 2(x30) + 4(x31) + 4(x32) + 6(x33) + 0(x34) + 0(x35) + 0(x36) + 0(x37) + -2(x38) + 0(x39) + 0(x40) + 4(x41) + 0(x42) + 0(x43) + 2(x44) + 2(x45) + 2(x46) + 0(x47) + 4(x48) + 2(x49) + -2(x50) + 2(x51) + 2(x52) + 2(x53) + -4(x54) + 4(x55) + 0(x56) + 0(x57) + -2(x58) + 0(x59) + -2(x60) + 2(x61) + -2(x62) + -2

Network for letter C:

-3(x0) + -1(x1) + 1(x2) + -1(x3) + 1(x4) + -1(x5) + 3(x6) + -1(x7) + 5(x8) + 3(x9) + 1(x10) + 1(x11) + 1(x12) + 3(x13) + 3(x14) + 1(x15) + 3(x16) + -1(x17) + 3(x18) + 1(x19) + 1(x20) + 3(x21) + 1(x22) + -1(x23) + 1(x24) + 1(x25) + 1(x26) + 1(x27) + 3(x28) + -3(x29) + -3(x30) + -1(x31) + -1(x32) + -1(x33) + 3(x34) + 3(x35) + -1(x36) + -1(x37) + -1(x38) + 1(x39) + -1(x40) + 1(x41) + 3(x42) + -

3(x43) + 3(x44) + 1(x45) + 3(x46) + -1(x47) + 1(x48) + -1(x49) + 1(x50) + 3(x51) + 3(x52) + 1(x53) + -1(x54) + 3(x55) + -5(x56) + -3(x57) + -1(x58) + 1(x59) + -1(x60) + -1(x61) + -1(x62) + -3

Network for letter D:

1(x0) + 1(x1) + -1(x2) + -1(x3) + 1(x4) + -5(x5) + -1(x6) + 1(x7) + 3(x8) + 3(x9) + 1(x10) + 3(x11) + 3(x12) + -1(x13) + 1(x14) + 3(x15) + 3(x16) + 1(x17) + 1(x18) + 1(x19) + 3(x20) + 1(x21) + 3(x22) + 1(x23) + 1(x24) + 1(x25) + 1(x26) + 3(x27) + 1(x28) + -1(x29) + -5(x30) + -1(x31) + -3(x32) + -3(x33) + 7(x34) + 1(x35) + 1(x36) + 1(x37) + -1(x38) + 1(x39) + -1(x40) + 3(x41) + 1(x42) + -1(x43) + 3(x44) + 3(x45) + 1(x46) + -1(x47) + 3(x48) + 1(x49) + -1(x50) + 3(x51) + 3(x52) + 3(x53) + 1(x54) + -1(x55) + -1(x56) + -1(x57) + -3(x58) + 1(x59) + -1(x60) + -5(x61) + -1(x62) + -3

Network for letter E:

3(x0) + 3(x1) + -1(x2) + 3(x3) + 5(x4) + -1(x5) + 7(x6) + 3(x7) + 1(x8) + 3(x9) + 1(x10) + -1(x11) + -1(x12) + -1(x13) + 1(x14) + 3(x15) + 3(x16) + -3(x17) + 1(x18) + 1(x19) + -5(x20) + 1(x21) + 3(x22) + -3(x23) + 7(x24) + 1(x25) + 1(x26) + -5(x27) + 1(x28) + 3(x29) + 1(x30) + 9(x31) + 1(x32) + -7(x33) + 3(x34) + 1(x35) + 1(x36) + 1(x37) + 1(x38) + 1(x39) + -1(x40) + -5(x41) + 1(x42) + -1(x43) + 3(x44) + 3(x45) + -3(x46) + -1(x47) + -5(x48) + 3(x49) + -3(x50) + 3(x51) + 3(x52) + 3(x53) + -7(x54) + -1(x55) + 1(x56) + 1(x57) + -3(x58) + 5(x59) + 3(x60) + -1(x61) + 9(x62) + -3

Network for letter J:

0(x0) + 0(x1) + -4(x2) + -2(x3) + 0(x4) + 2(x5) + 4(x6) + 0(x7) + 0(x8) + 2(x9) + 0(x10) + 2(x11) + 2(x12) + 0(x13) + -2(x14) + 2(x15) + 2(x16) + 0(x17) + 2(x18) + 4(x19) + -2(x20) + -2(x21) + 2(x22) + 0(x23) + 2(x24) + 0(x25) + 4(x26) + 0(x27) + -2(x28) + 0(x29) + -2(x30) + 0(x31) + -2(x32) + 2(x33) + 2(x34) + -2(x35) + 0(x36) + 0(x37) + 0(x38) + 0(x39) + 2(x40) + 0(x41) + -2(x42) + 2(x43) + 2(x44) + 2(x45) + 2(x46) + 2(x47) + 0(x48) + 0(x49) + 0(x50) + 2(x51) + 2(x52) + 2(x53) + 0(x54) + 0(x55) + -2(x56) + -2(x57) + -2(x58) + 0(x59) + -2(x60) + -2(x61) + 0(x62) + -2

Network for letter K:

2(x0) + 0(x1) + -2(x2) + -4(x3) + -2(x4) + 2(x5) + 2(x6) + 0(x7) + 2(x8) + 0(x9) + -2(x10) + 4(x11) + 0(x12) + -2(x13) + 0(x14) + 2(x15) + 0(x16) + 2(x17) + 0(x18) + 0(x19) + -2(x20) + 0(x21) + 2(x22) + 2(x23) + 0(x24) + -2(x25) + 0(x26) + -2(x27) + 0(x28) + 2(x29) + -2(x30) + -2(x31) + -4(x32) + -2(x33) + 0(x34) + 0(x35) + 0(x36) + 0(x37) + 0(x38) + -2(x39) + -2(x40) + -2(x41) + 0(x42) + 0(x43)

+ 0(x44) + 2(x45) + 2(x46) + -2(x47) + -2(x48) + 0(x49) + 0(x50) + 0(x51) + 0(x52) + 2(x53) + 0(x54) + -2(x55) + 0(x56) + -2(x57) + -2(x58) + -2(x59) + -4(x60) + 0(x61) + 0(x62) + 0

Training converged after 6 epochs.

Number of test images: 21

==========

Test image 1

Stage 1: A

Stage 2: A

Stage 3: A

==========

Test image 2

Stage 1: Not A

Stage 2: B

Stage 3: B

==========

Test image 3

Stage 1: Not A

Stage 2: Unknown / Uncertain

Stage 3: Unknown / Uncertain

==========

Test image 4

Stage 1: Not A

Stage 2: D

Stage 3: D

==========

Test image 5

Stage 1: Not A

Stage 2: E

Stage 3: E

==========

Test image 6

Stage 1: Not A

Stage 2: J

Stage 3: J

==========

Test image 7

Stage 1: Not A

Stage 2: K

Stage 3: K

==========

Test image 8

Stage 1: A

Stage 2: Unknown / Uncertain

Stage 3: Unknown / Uncertain

==========

Test image 9

Stage 1: Not A

Stage 2: B

Stage 3: B

==========

Test image 10

Stage 1: Not A

Stage 2: C

Stage 3: C

==========

Test image 11

Stage 1: Not A

Stage 2: D

Stage 3: D

==========

Test image 12

Stage 1: Not A

Stage 2: E

Stage 3: E

==========

Test image 13

Stage 1: Not A

Stage 2: Unknown / Uncertain

Stage 3: Unknown / Uncertain

==========

Test image 14

Stage 1: Not A

Stage 2: K

Stage 3: K

==========

Test image 15

Stage 1: A

Stage 2: A

Stage 3: A

==========

Test image 16

Stage 1: Not A

Stage 2: Unknown / Uncertain

Stage 3: Unknown / Uncertain

==========

Test image 17

Stage 1: Not A

Stage 2: C

Stage 3: C

==========

Test image 18

Stage 1: Not A

Stage 2: Unknown / Uncertain

Stage 3: Unknown / Uncertain

==========

Test image 19

Stage 1: Not A

Stage 2: Unknown / Uncertain

Stage 3: Unknown / Uncertain

==========

Test image 20

Stage 1: Not A

Stage 2: Unknown / Uncertain

Stage 3: Unknown / Uncertain

==========

Test image 21

Stage 1: Not A

Stage 2: K

Stage 3: K

==========

FINAL RESULTS

Stage 1:

A: 3

Not A: 18

Uncertain: 0

Stage 2:

A: 2

B: 2

C: 2

D: 2

E: 2

J: 1

K: 3

Unknown: 7

Stage 3:

A: 2

B: 2

C: 2

D: 2

E: 2

J: 1

K: 3

Unknown: 7

Comparison:

Agreement: 21/21

Agreement percentage: 100.00%